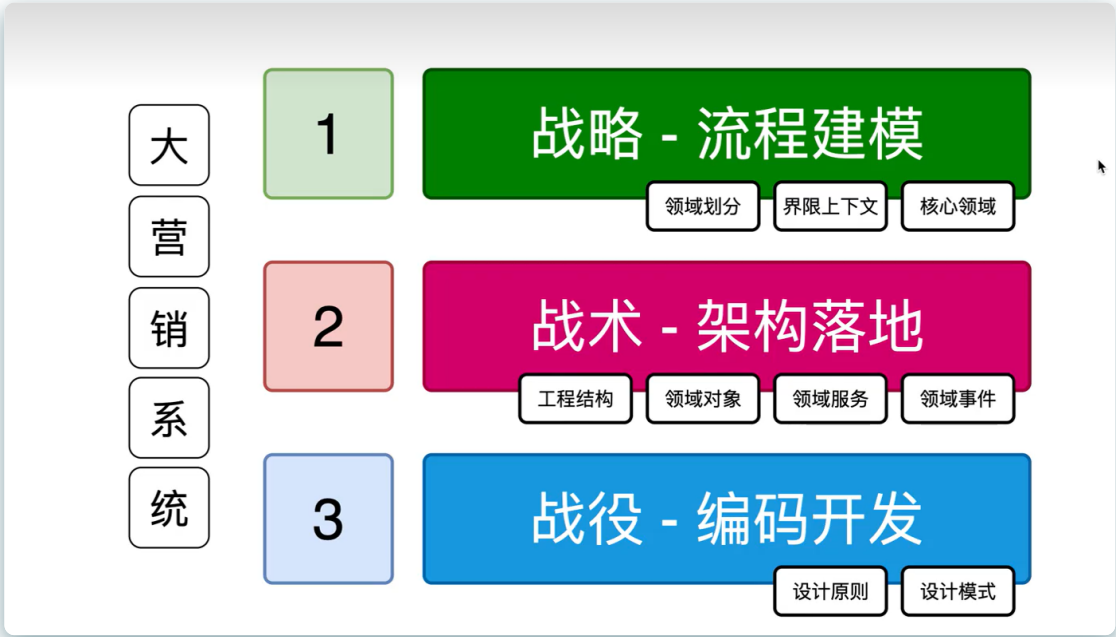
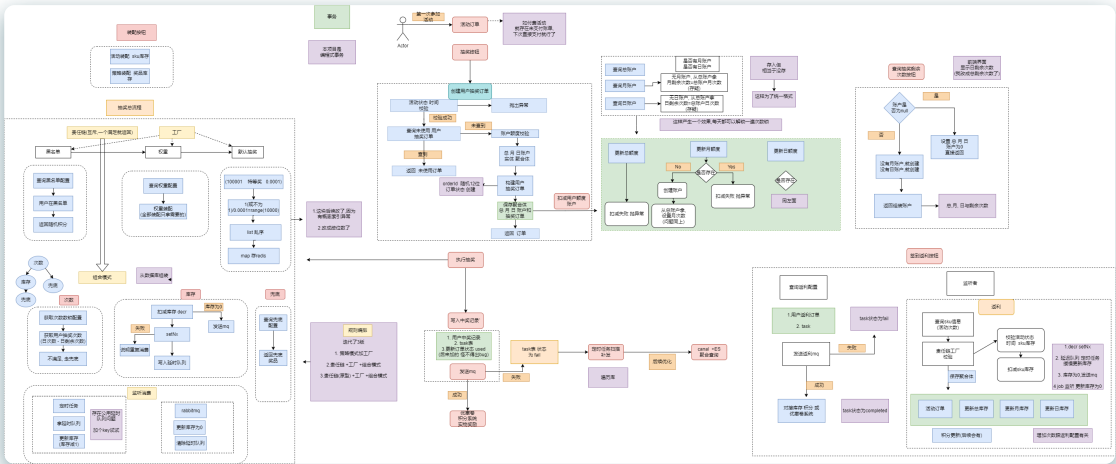


DDD落地



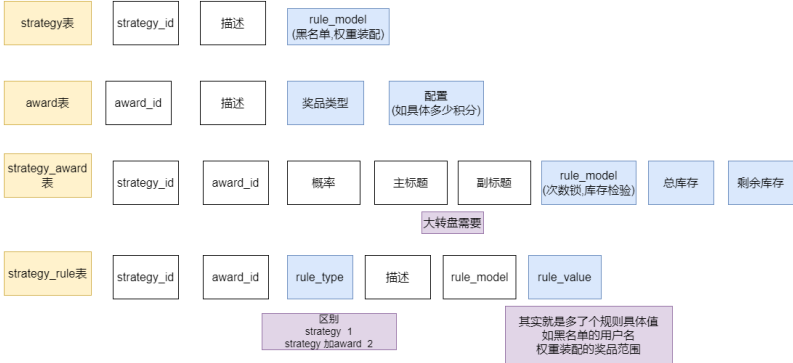
总流程



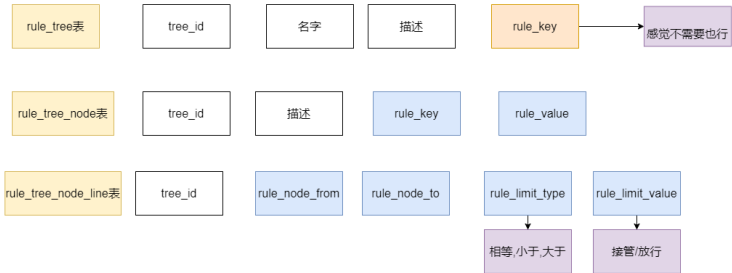
库表设计

需要注意的字段 难理解的字段 补充说明 标题

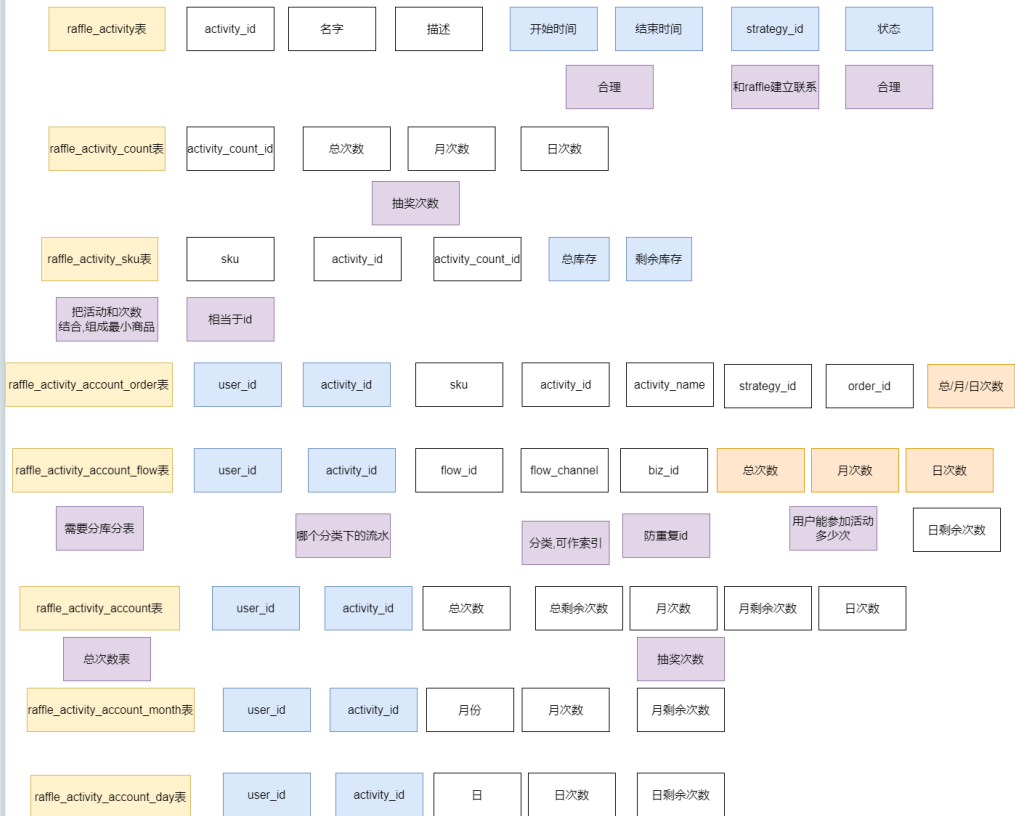
抽奖功能



规则树

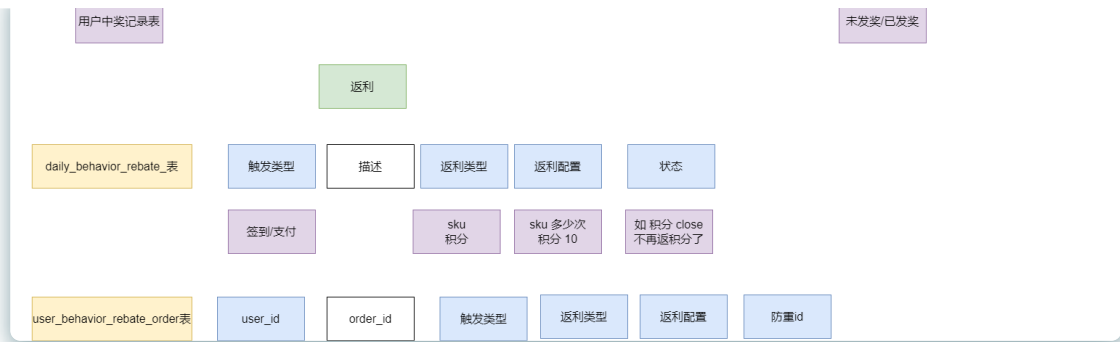


活动



用户





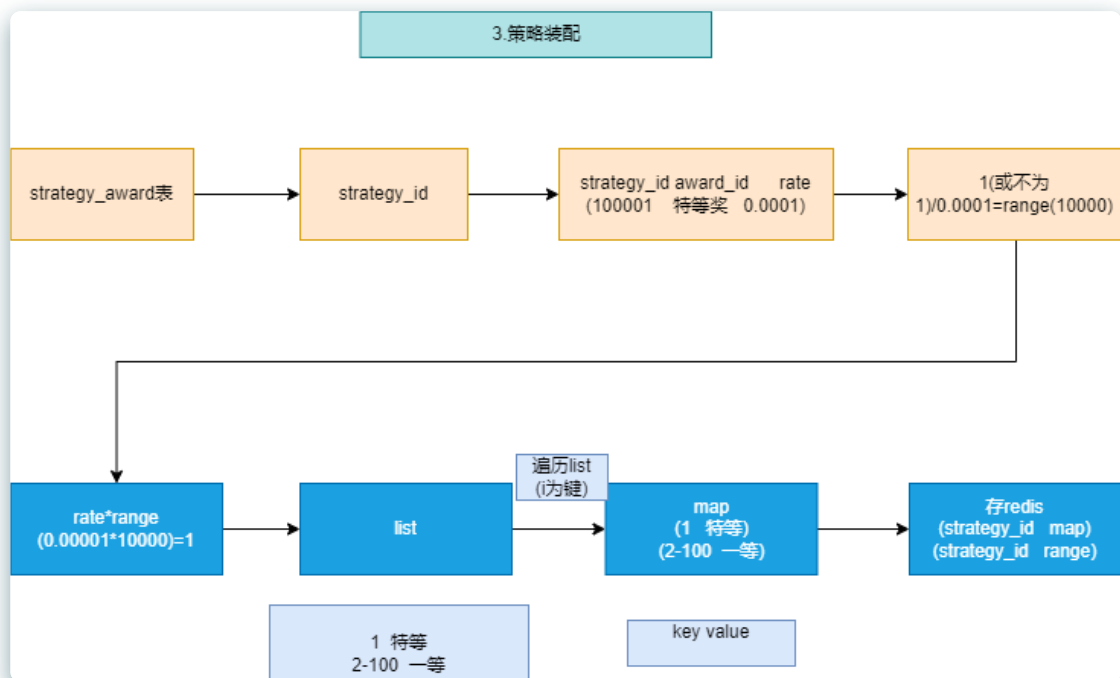
缺一个活动订单表

返利配置

返利流水

抽奖概率

✓ 全部装配



这块代码修改了

原代码

```
// 3. 用 1 % 0.0001 获得概率范围, 百分位、千分位、万分位 向上取整
BigDecimal rateRange = totalAwardRate.divide(minAwardRate, 0, RoundingMode.CEILING);

// 4. 生成策略奖品概率查找表「这里指需要在list集合中, 存放上对应的奖品占位即可, 占位越多等于概率越高」
List<Integer> strategyAwardSearchRateTables = new ArrayList<>(rateRange.intValue());
for (StrategyAwardEntity strategyAward : strategyAwardEntities) {
    Integer awardId = strategyAward.getAwardId();
    BigDecimal awardRate = strategyAward.getAwardRate();
    // 计算出每个概率值需要存放到查找表的数量, 循环填充 向上取整
    for (int i = 0; i < rateRange.multiply(awardRate).setScale(0, RoundingMode.CEILING).intValue(); i++) {
        strategyAwardSearchRateTables.add(awardId);
    }
}
```

新代码

```
// 2. 循环计算找到概率范围值
BigDecimal rateRange = BigDecimal.valueOf(convert(minAwardRate.doubleValue()));
```

为什么要获取概率总和, 每个策略的总和不一定一定是100吗? 难道不是?

奖品总概率可以大于1,如 总概率/最小概率 $1.01/0.0001=100100$

10000份随机积分,100份一/二/三等奖

总概率小于1同理,实际概率是动态分配的,这也算一种健壮性,数据库概率总和不为1也不会出错

为什么表示总的概率值除最小概率值, 不保留小数, 并向上取整, 可以取得概率范围?

首先,奖品数不能是小数,一般情况,如最小概率 0.0001 $1.01/0.0001=100100$
 $1/0.0001=100100$ 得到的都是整数,向上取整与否都行

其次 考虑 如最小概率 0.00011 $1/0.00011=9090.9090$

1.都向上取整 9091

$9091*0.99989=9089.999$ 取成9090

$9091*0.00011=1.00001$ 取成2 加起来9092

这种情况有极小概率超出索引 也算是极端情况 小bug

2都向下取整

$9090 * 0.99989 = 9089.0001$ 取成9089

$9090 * 0.00011 = 0.999$ 取成0

这种情况没超出索引 但概率小的奖品直接没了,不合适`

解决办法

1. 四舍五入就能解决(没验证)

2.xfg的新代码 只根据小数位数得到 range 也不再算概率总和 用总概率/最小概率

```
/**
 * 转换计算, 只根据小数位数来计算。如【0.01返回100】、【0.009返回1000】、【0.0018返回10000】
 */
private double convert(double min) {
    if(0 == min) return 1D;

    double current = min;
    double max = 1;
    while (current < 1) {
        current = current * 10;
        max = max * 10;
    }
    return max;
}
```

但这里 max=1 这要求你数据库概率总和要检查下 是否为1

假如概率总和 为1.01 range范围10000 实际奖品范围10100 超出索引

假如概率总和 为0.99 range范围10000 实际奖品范围9900 一部分为空值

当然也能解决,null默认为未中奖

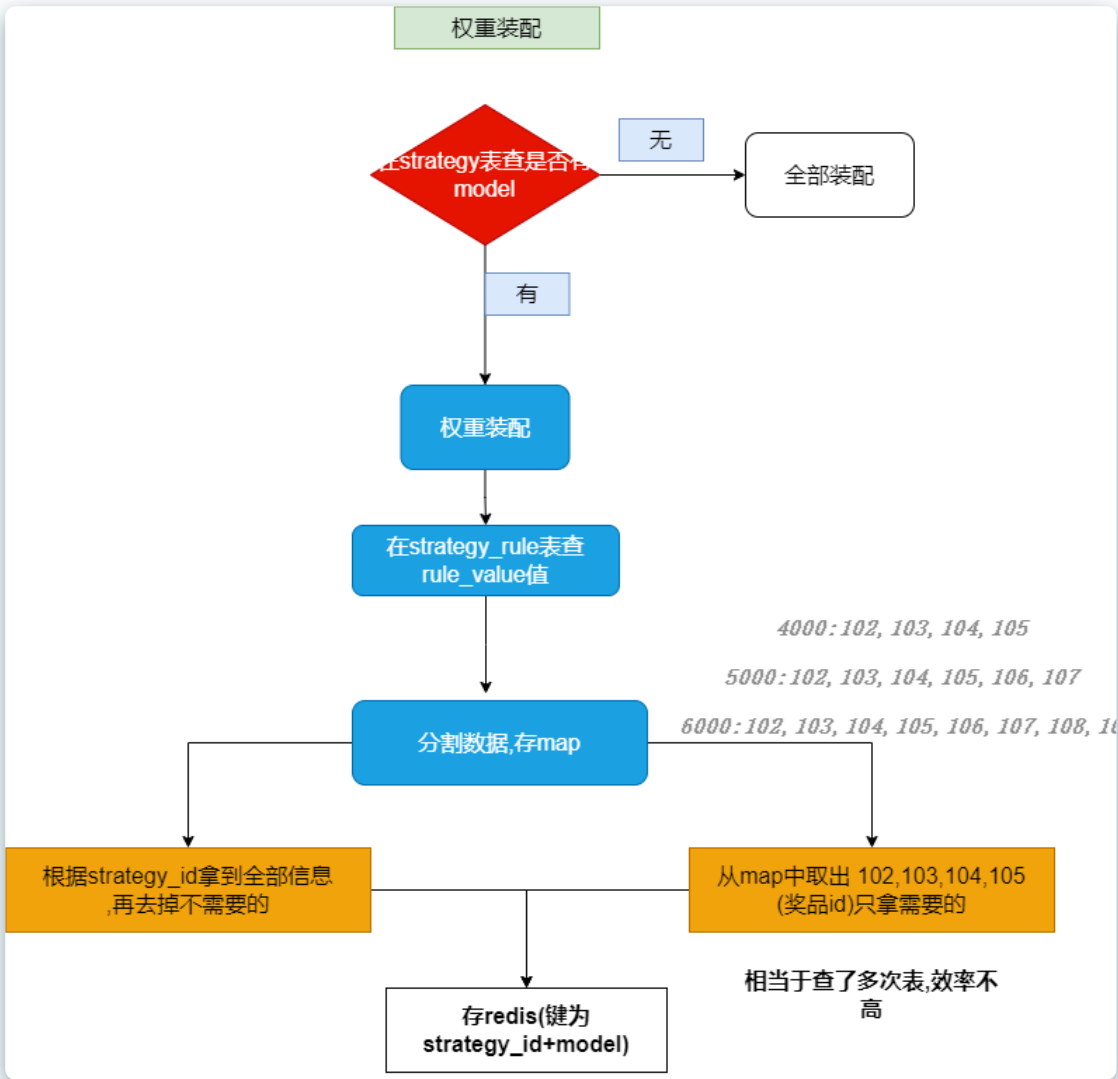
另外,也可尝试把max换成总概率

假如概率总和 为1.01 range范围10100 实际奖品范围10100

假如概率总和 为0.99 range范围9900 实际奖品范围9900

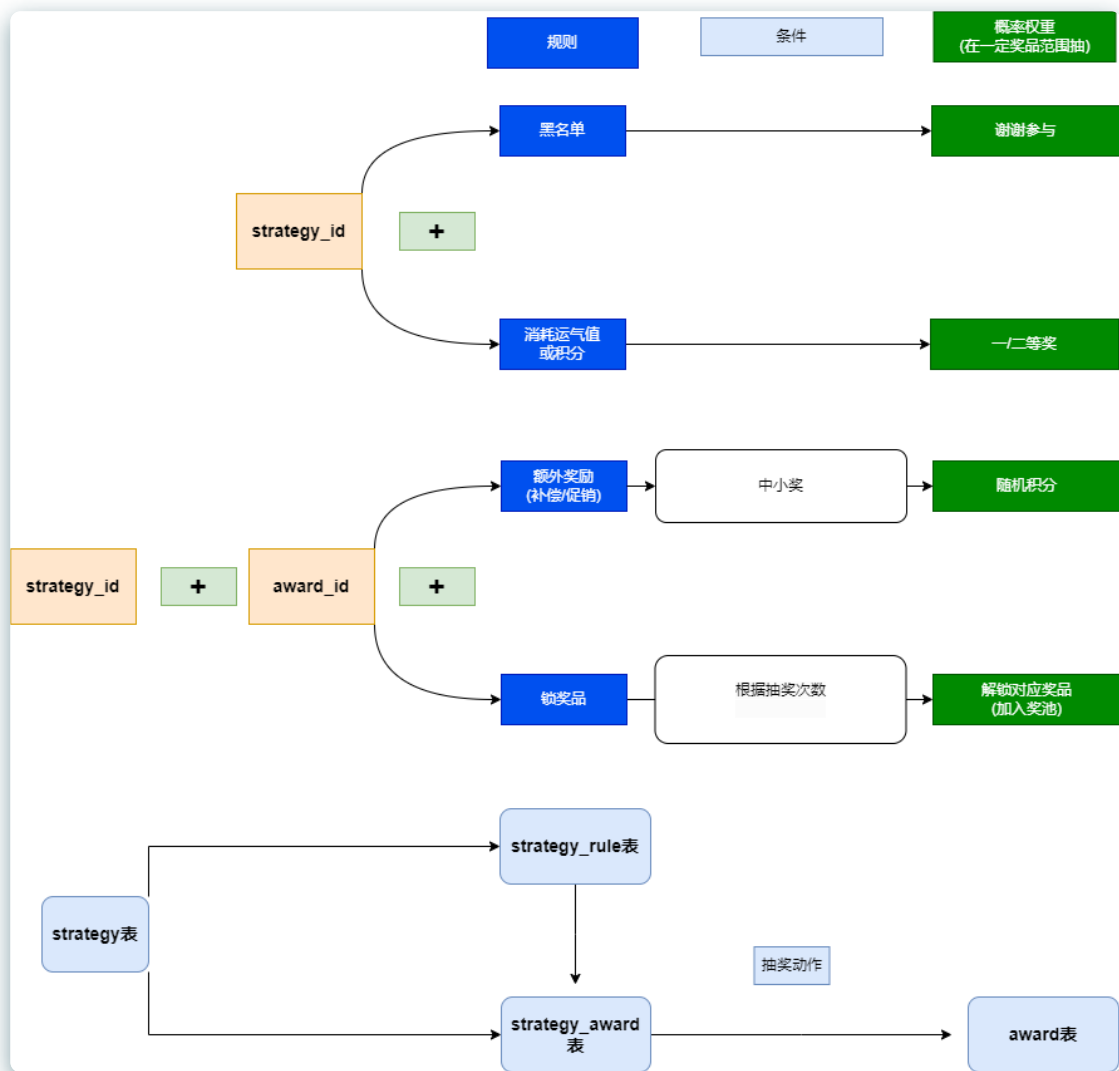
?我改成 总概率 反而 索引不对 库总概率0.23,代码用max=1却没错 不清楚

权重装配

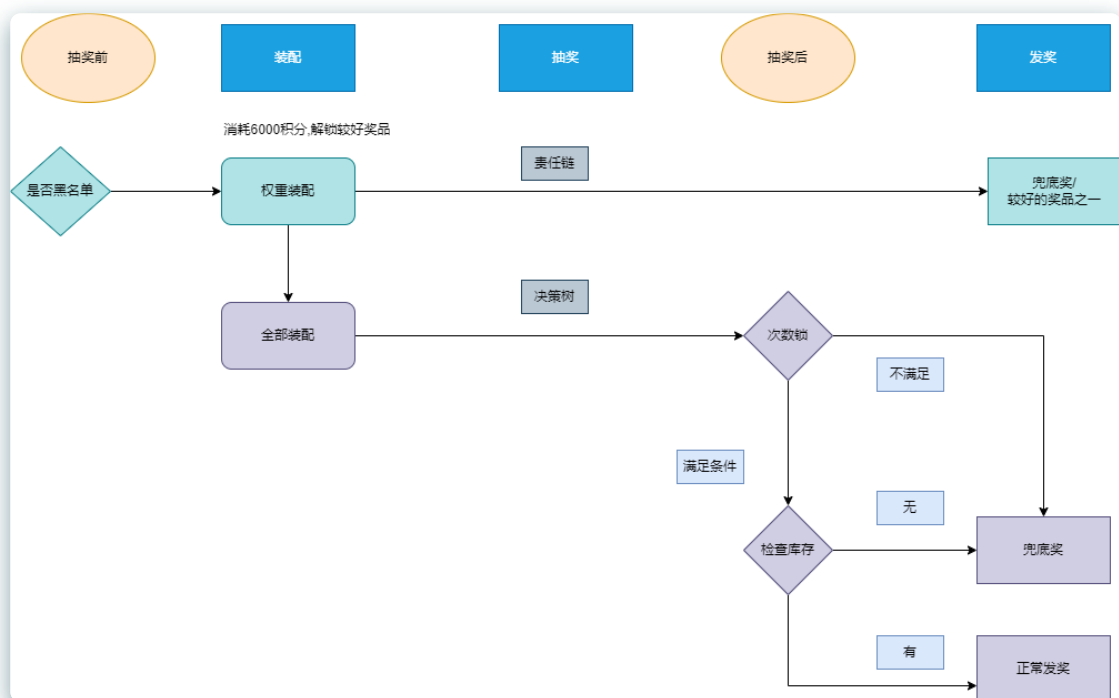


规则编排

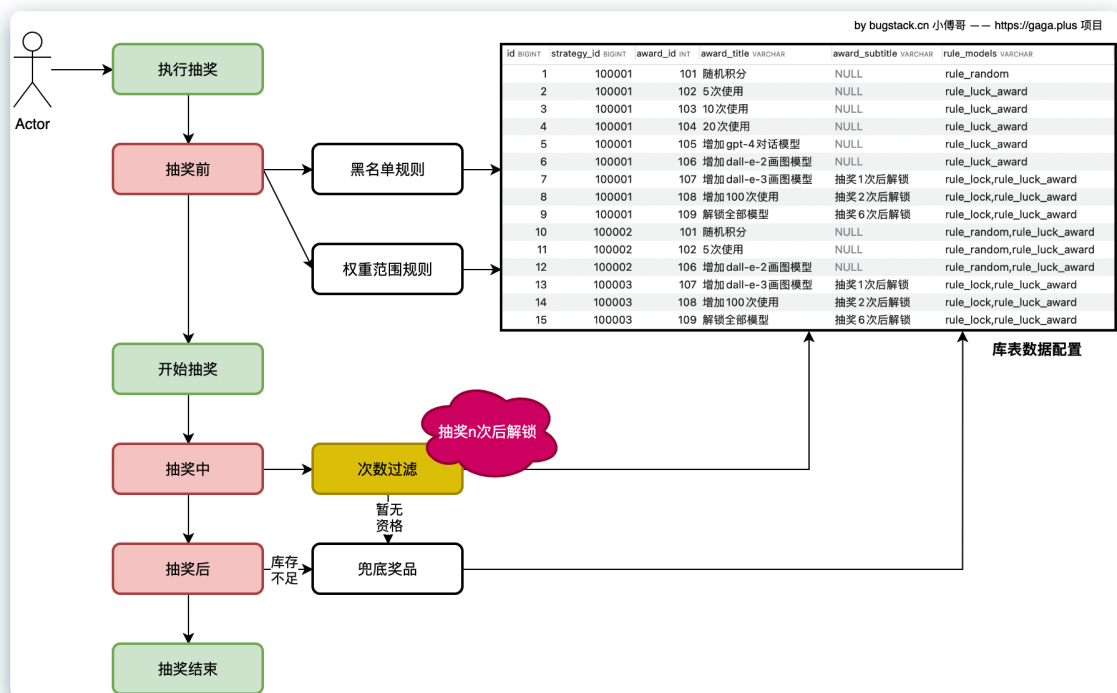
✓ 规则



✓ 规则总览



图二



✓ 规则过滤器

后面直接重构没了

自定义注解

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
public @interface LogicStrategy {

    DefaultLogicFactory.LogicModel logicMode();

}
```

为什么自定义注解?

平时我们写代码是不是喜欢在类上加注解 这个自定义注解 就是方便我们在其他类加注解 自定义注解的内容 是字段 如rule_weightet 把它加到rule_weightet规则过滤器类上面

每个子类加注解

```
@LogicStrategy(logicMode = DefaultLogicFactory.LogicModel.RULE_LOCK)
public class RuleLockLogicFilter implements ILogicFilter<RuleActionEntity.RaffleCenterEntity> {
```

factory装配

```
@Service
public class DefaultLogicFactory {

    //装配
    public Map<String, ILogicFilter<?>> logicFilterMap = new ConcurrentHashMap<>();

    // List<ILogicFilter<?>> logicFilters
    public DefaultLogicFactory(List<ILogicFilter<?>> logicFilters) {
        logicFilters.forEach(logic -> {
            LogicStrategy strategy = AnnotationUtils.findAnnotation(logic.getClass(), LogicStrategy.class);
            if (null != strategy) {
                logicFilterMap.put(strategy.logicMode().getCode(), logic);
            }
        });
    }

    public <T extends RuleActionEntity.RaffleEntity> Map<String, ILogicFilter<T>> openLogicFilter() {
        return (Map<String, ILogicFilter<T>>) (Map<?, ?>) logicFilterMap;
    }

    @Getter
    @AllArgsConstructor
    public enum LogicModel {

        RULE_WIGHT("rule_weight", "【抽奖前规则】根据抽奖权重返回可抽奖范围KEY", "before"),
        RULE_BLACKLIST("rule_blacklist", "【抽奖前规则】黑名单规则过滤, 命中黑名单则直接返回",
"before"),
        RULE_LOCK("rule_lock", "【抽奖中规则】抽奖n次后, 对应奖品可解锁抽奖", "center"),
        RULE_LUCK_AWARD("rule_luck_award", "【抽奖后规则】抽奖n次后, 对应奖品可解锁抽奖", "after"),
        ;

        private final String code;
        private final String info;
        private final String type;

    }
}
```

List<ILogicFilter<?>> logicFilters 为什么有值 那是怎么进list的?

这个过程是在应用程序启动时进行的，通常是在Spring上下文初始化的早期阶段。具体的方法和周期如下：

1. 扫描和解析组件：Spring会扫描应用程序上下文中的所有bean定义，包括通过注解（如@Component、@Service等）或XML配置进行声明的bean。
2. 创建bean实例：Spring会根据bean定义创建相应的bean实例。对于通过注解进行声明的bean，Spring使用反射机制实例化对象；对于通过XML配置进行声明的bean，Spring会根据配置信息创建对象。
3. 处理依赖注入：在创建bean实例的过程中，当Spring遇到需要依赖注入的字段或构造方法参数时，它会查找匹配的依赖并进行注入。对于List<ILogicFilter<?>>>字段，Spring会查找所有实现ILogicFilter接口的bean，并将它们添加到列表中。
4. 完成初始化：在完成所有bean的依赖注入后，Spring上下文会触发一系列的生命周期回调方法（如@PostConstruct注解标记的方法），以便执行一些额外的初始化操作。

需要注意的是，具体的方法和周期可能会受到不同的Spring配置和使用的注解的影响。但通常情况下，上述的步骤是在Spring应用程序的启动阶段自动发生的。

然后这样放进map里 logicFilterMap.put(strategy.logicMode().getCode(), logic);

接管或放行

// 用户抽奖次数大于规则限定值，规则放行

```
if (userRaffleCount >= raffleCount) {  
  
    return RuleActionEntity.<RuleActionEntity.RaffleCenterEntity>builder()  
  
        .code(RuleLogicCheckTypeVO.ALLOW.getCode())  
  
        .info(RuleLogicCheckTypeVO.ALLOW.getInfo())  
  
        .build();  
  
}
```

// 用户抽奖次数小于规则限定值，规则拦截

```
return RuleActionEntity.<RuleActionEntity.RaffleCenterEntity>builder()  
  
    .code(RuleLogicCheckTypeVO.TAKE_OVER.getCode())  
  
    .info(RuleLogicCheckTypeVO.TAKE_OVER.getInfo())  
  
    .build();
```

```
}
```

✓ 责任链

前言

链表原理有待加强

A说:责任链整条链路应该是固定的, 不应该这样动态组装, 而且他说你的工厂是被spring接管的, 然后第二个人生成的链会把第一个人的覆盖了

后面原型模式 每一个装配的责任链都有自己的id「策略id」管理, 是不会被覆盖的。

还可以优化, 把创建的责任链本地缓存起来, 后面的调用直接使用即可

可用 Guava ,redis map为(strategy_id, 责任链)

几个节点

BlackListLogicChain

直接返回奖品id

101:user001,user002,user003

```
ruleValue.split(,)
```

RuleWeightLogicChain

6000:102,103,104,105,106,107,108,109

6000 条件值 执行权重抽奖

DefaultLogicChain

执行抽奖就行

固定长度创建

就是不根据数据库, 在程序中手动创建固定长度责任链, 比如, 固定 rule_weight, back_list 默认责任链这三个

动态创建

本项目就是动态创建, 根据数据库中strategy有rule_weight, back_list组装, 最后添加一个默认责任链 如strategy只有rule_weight 责任链为rule_weight 默认责任链 所以是动态的

原本代码

```
public DefaultChainFactory(Map<String, ILogicChain> logicChainMap) {  
    this.logicChainMap = logicChainMap;  
    this.strategyRepository = strategyRepository;  
}
```

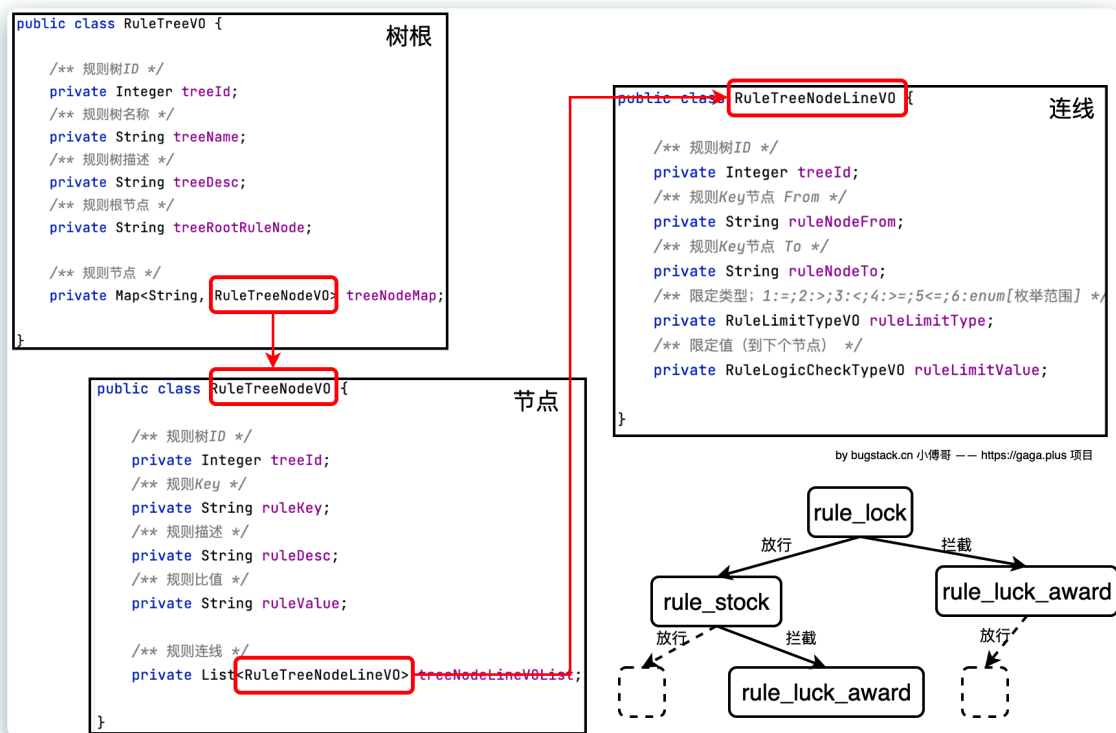
都在logicChainMap , 然后从数据库取出字段, 从map拿 组装责任链

原型

```
// 按照配置顺序装填用户配置的责任链; rule_blacklist、 rule_weight 「注意此数据从Redis缓存中获取, 如果更新库表, 记得在测试阶段手动处理缓存」  
ILogicChain logicChain = applicationContext.getBean(ruleModels[0], ILogicChain.class); // bean  
名字 bean类型  
ILogicChain current = logicChain;  
for (int i = 1; i < ruleModels.length; i++) {  
    ILogicChain nextChain = applicationContext.getBean(ruleModels[i], ILogicChain.class);  
    current = current.appendNext(nextChain);  
}
```

面试模拟: 我迭代了3版责任链 1 固定长度 手动组装 2 动态组装 3 原型

✓ 组合模式---规则树



这个库表很巧妙 ,组装规则树 还能扩展为森林

三个树节点

RuleLockLogicTreeNode

判断下抽奖次数 接管或放行

RuleStockLogicTreeNode

扣减库存是否成功 接管或放行

RuleLuckAwardLogicTreeNode

接管

不超卖库存

和 后面的sku库存不超卖原理一致 就只在这里细讲了

decr

(互斥,一般不会产生超卖)

decr 命令在 Redis 中的主要好处包括:

原子性操作: decr 是一个原子操作, 这意味着即使在高并发环境下, 对于同一 key 的递减操作也是互斥的, 不会出现竞态条件 (race condition), 保证了数据的一致性和完整性。

高性能计算: 在 Redis 这种内存数据库中, 执行 decr 操作非常迅速, 相比起每次都需要从数据库读取数据、修改后再写回的传统数据库操作, Redis 可以显著提升计数器类业务场景的性能。

简化开发: 在诸如统计访问量、商品库存扣减、积分系统中扣除积分等应用场景下, 直接调用 decr 命令可以大大简化开发工作, 无需编写额外的加减逻辑和事务处理代码。

空间效率: 由于 Redis 内部实现了对整数值的优化存储, 使用 decr 更新整数值时, 仅需修改内存中的数值, 不需要像文本或者其他复杂类型那样重新分配存储空间。

setnx 加锁兜底

setnx 加锁是一种兜底手段, 避免后续有库存的恢复, (redis挂了,人为恢复)导致库存从96消耗后又回到了98重复消费。所以对于每个key加锁, 98、97、96... 即使有恢复库存也不会导致超卖。【setnx 在 redisson 是用 trySet 实现】

关于 setNx redis节点挂 延迟队列还没有来得及更新库存 此时数据实际库存多了, 1. 然后把数据库库存 初始化到redis(redis无这条库存),原来redis库存是95(挂掉之前) 从数据库初始化到redis(98)就重复消费了 setNx锁流水啊 流水记录有 99 98 97 96 95 你又从98扣成97 明显和流水不符 人工操作一下 不能超卖

2.redis有这条库存 不能初始化 但redis库存 和数据库存不一致了(数据库始终多) ,setNx的流水可以做对照

```
redissonClient.getBucket(key).trySet("lock")
```

在 Redisson 中，Bucket 类用于操作单个值的 Redis 键，其 trySet(V value) 方法尝试将桶 (bucket) 的值设置为给定值，如果 bucket 之前没有值，则设置成功并返回 true；如果已经有值，则设置失败并返回 false。

要是 redis记录全没 连流水(98 97)都丢了咋办？

一般生产中都是主从的配置，如果都没了，全都没了，先嘎了redis运维。没有办法保证全年0问题，但出问题就，运营、产品、测试、研发、运维等，各司其职。实际中大部分不是环境这些问题，而是运营配置参数问题，即使是事故，也都没有什么科技含量的事故，都是些字段不对、值没判空、接口不符合预期等。

延迟消费队列

```
// 生成消费队列
```

```
@Override
public void awardStockConsumeSendQueue(StrategyAwardStockKeyVO strategyAwardStockKeyVO) {
    String cashKey = Constants.RedisKey.STRATEGY_AWARD_COUNT_QUERY_KEY;
    RBlockingQueue<Object> blockingQueue = redisService.getBlockingQueue(cashKey);
    RDelayedQueue<Object> delayedQueue = redisService.getDelayedQueue(blockingQueue);
    delayedQueue.offer(strategyAwardStockKeyVO, 3, TimeUnit.SECONDS);
}
```

延迟队列加上 定时任务 就是双重减缓 减小数据库压力

消费队列是在RuleStockLogicTreeNode(规则树)调用生成，!!!相当于权重装配没有走库存,不进行库存扣减

定时任务

Cron 表达式 "0/5 * * * * ?" 解释如下：

第一位 0：表示分钟域的值，这里意味着从第 0 分钟开始，每隔 5 分钟执行一次（即 0, 5, 10, 15, ...）。

接下来的四个星号 * 分别代表小时域、月份中的日期域、月份域、星期域，均表示任意值，即不作任何限制。

最后一个问号？在某些 Cron 表达式实现中表示月份中的日期（day-of-month）和星期（day-of-week）两个字段不必同时满足，只要其中之一满足条件即可执行任务。但在

Spring 的 @Scheduled 注解中，它通常用来兼容 Quartz Cron 表达式，并无特殊含义。因此，整个表达式意味着该方法将会每 5 分钟执行一次，不受日期、小时或其他日历因素影响。

1.拿消费队列

```
redisService.getBlockingQueue(cashKey)
```

而不是redisService.getDelayedQueue(cashKey);

2.更新数据库库存

部分代码解释

```
@Override
public Long getAtomicLong(String key) {
return redissonClient.getAtomicLong(key).get();
}
```

该函数通过Redisson客户端获取具有指定键的原子长整型对象，并返回该对象的当前值。

```
@Override
public void setAtomicLong(String cacheKey, Integer awardCount) {
redissonClient.getAtomicLong(cacheKey).set(awardCount);
}
```

该函数使用Redisson客户端获取具有给定缓存键的AtomicLong对象，并将指定的奖品数量设置为该AtomicLong对象的值。

!decr setNx都是在这一步

```
public Boolean subtractionAwardStock(String cacheKey) {
//扣减库存(存的是总库存,为了加锁)
long surplus = redisService.decr(cacheKey);
if (surplus < 0) {
redisService.setValue(cacheKey,0);
return false;
};
String lockKey = cacheKey + Constants.UNDERLINE + surplus;
Boolean lock= redisService.setNx(lockKey);
if (!lock) {
log.info("策略奖品库存加锁失败, ", lockKey);
}

return lock;
}
```

✓ 发送mq

好处: 当redis中库存为0, 但延迟队列还在 还要缓慢更新数据库库存 发送mq, 直接更新数据库库存为0, 清掉延迟队列, 相当于少了n次数据库连接, 又保证了最终一致

思考

1.消费队列是在RuleStockLogicTreeNode(规则树)调用生成,

!!!相当于权重装配没有走规则树,不进行库存扣减(需要人工统计)

2.为什么setNx加锁锁的总库存而不是剩余库存?

锁的是库存编号,锁一个总库存,就产生竞争了

db-router

为什么用 用户id进行分库分表

仔细研究库表 可以发现,那些需要分库分表的 都包含用户id 用户id可以贯穿全部 而其他字段不能做到

学习目标: 实现和使用分库分表 (引用 liuc大佬)

①垂直拆分: 可以按照业务将表分到不同的数据库

②水平拆分: 单表数据量过大, 进行水平拆分(根据用户id), 将同一个表的数据散列到不同的数据库中, 就是本节要实现的

1、 annotation DBRouter:

路由注解, 标记分库分表依据的字段 DBRouterStrategy:

路由策略注解，标记是否分表

2、config.DataSourceAutoConfig:

①dataSourceMap：接收多数据源

②defaultDataSourceConfig：保存默认数据源配置

连接哪个数据库 表 账户 密码

③setEnvironment方法：解析application.yml中mini-db-router.jdbc.datasource.下的配置信息，包括分库数量dbCount、分表数量tbCount、路由字段routerKey，循环分库，取出每个库的配置信息，包括driver-class-name、url、username、password，将这些信息都存入dataSourceMap，获取default默认数据源，存入defaultDataSourceConfig

④dataSource方法：循环dataSourceMap将多数据源配置信息存入DynamicDataSource，该方法会将实例化对象交由Spring管理，并且DynamicDataSource继承了AbstractRoutingDataSource，可以存放和读取相应的具体调用的数据源信息。

分库关键:

3、dynamic.DynamicDataSource：动态数据源获取，每当切换数据源，都要从ThreadLocal中获取数据源的key

```
public class DynamicDataSource extends AbstractRoutingDataSource {  
  
    @Value("${mini-db-router.jdbc.datasource.default}")  
    private String defaultDataSource;  
  
    @Override  
    protected Object determineCurrentLookupKey() {  
        if(DBContextHolder.getDBKey() == null){  
            return defaultDataSource;  
        }else {  
            return "db"+DBContextHolder.getDBKey();  
        }  
    }  
  
}
```

分表关键

4、dynamic.DynamicMybatisPlugin：Mybatis 拦截器，通过对 SQL 语句的拦截处理，修改分表信息，首先定义正则表达式用来匹配SQL，获取StatementHandler，从中获取类上的注解@DBRouterStrategy，如果为空，说明不需要分表，否则需要分表，将表名拼接成
_01

5、strategy ①IDBRouterStrategy: 定义路由策略要执行的逻辑接口

②DBRouterStrategyHashCode: 实现上面的接口, 计算数据库路由, 首先提取库表乘积作为散列的元素长度, 使用扰动函数计算散列索引, 根据散列索引计算库表索引, 将计算出的库表索引放入ThreadLocal, 方便数据源切换时以及Mybatis拦截器使用

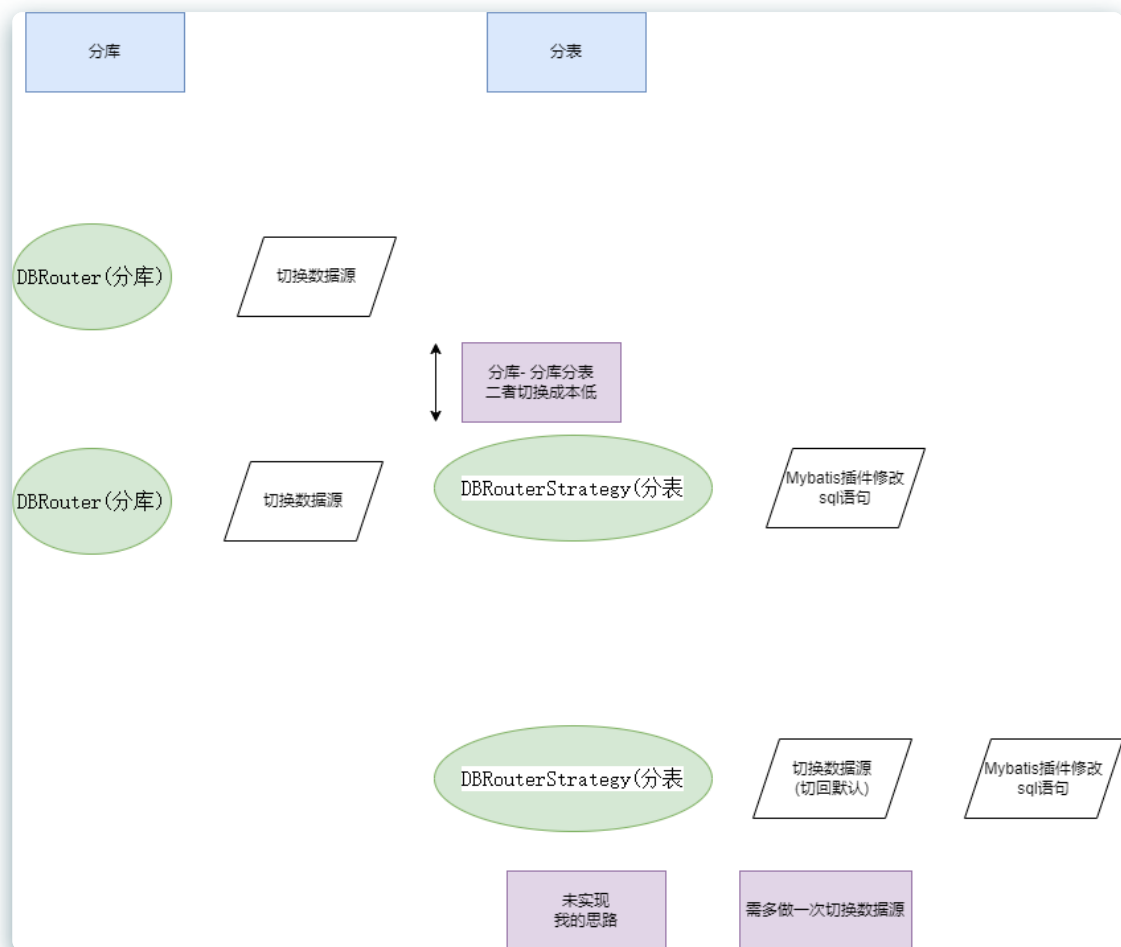
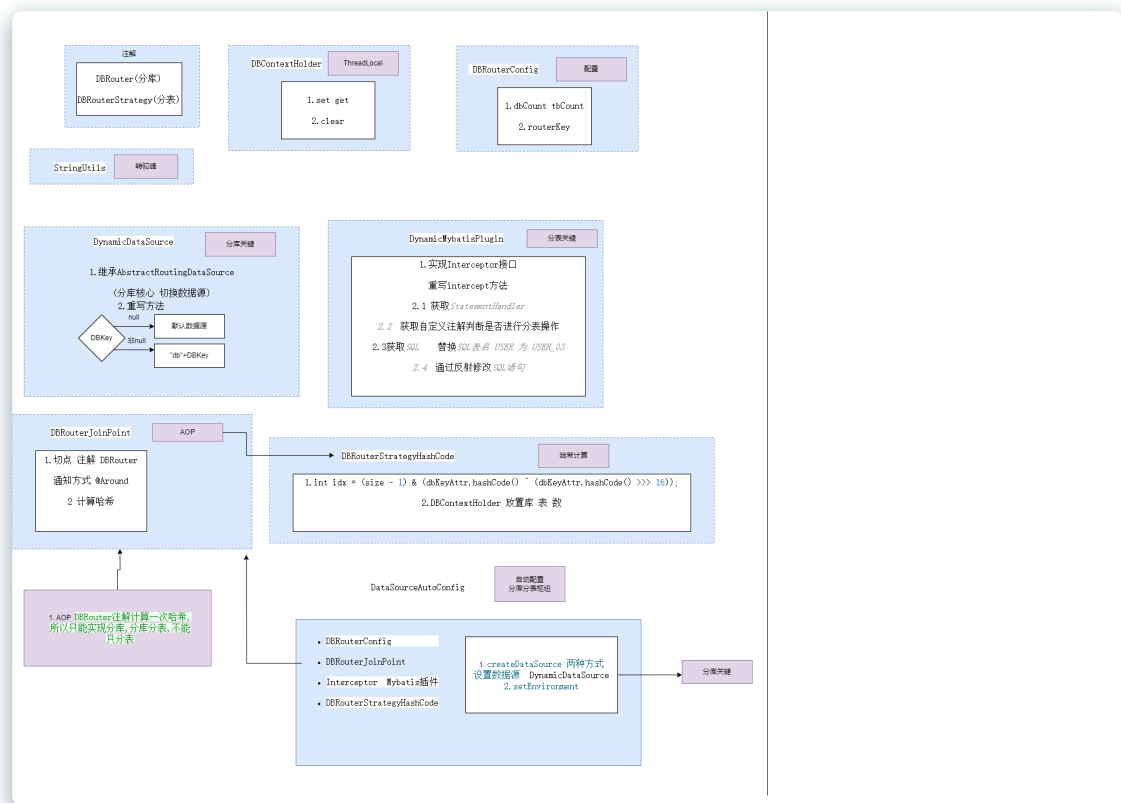
6、router ①DBContextHolder: 存储计算出的库表索引, 对外提供调用方法

②DBRouterConfig: 从application.yml中获取多数据路由配置信息, 包括dbCount、tbCount、routerKey

③DBRouterJoinPoint: 数据路由切面, 通过自定义注解的方式, 拦截被切面的方法, 进行分库分表的路由计算



流程



雪崩测试

从4库8表 到8库16 数据变化范围是否为 50%

Apache POI库

存excel表 行库 列表 值 数据量

计算哈希

```
public void doRouter(String dbKeyAttr) {
    int size = dbRouterConfig.getDbCount() * dbRouterConfig.getTbCount();
    // 扰动函数; 在 JDK 的 HashMap 中, 对于一个元素的存放, 需要进行哈希散列。而为了让散列更加均匀, 所以添加了扰动函数
    int idx = (size - 1) & (dbKeyAttr.hashCode() ^ (dbKeyAttr.hashCode() >>> 16));
    // 库表索引; 相当于是把一个长条的桶, 切割成段, 对应分库分表中的库编号和表编号
    // 公式目的: 8个位置, 计算出来的是位置在5 那么你怎么知道5是在2库1表
    int dbIdx = idx / dbRouterConfig.getTbCount() + 1;
    int tbIdx = idx - dbRouterConfig.getTbCount() * (dbIdx - 1);
    // 设置到 ThreadLocal
    DBContextHolder.setDbKey(String.format("%02d", dbIdx));
    DBContextHolder.setDbKey(String.format("%03d", tbIdx));
    logger.debug("数据库路由 dbIdx: {} tbIdx: {}", dbIdx, tbIdx);
}
```

对idx的计算范围表示疑问 是1-8吗?

注解

```
@Target({ElementType.METHOD, ElementType.TYPE})
public @interface DBRouterStrategy {

    boolean splitTable() default false;
}
```

@Retention(RetentionPolicy.RUNTIME) 指定该注解将在运行时被 JVM 保留, 从而可以通过反射访问。

@Target({ElementType.METHOD, ElementType.TYPE}) 声明此注解可以应用于方法 (ElementType.METHOD) 和类或接口 (ElementType.TYPE)。

配置解析

定义一个Java类来持有你的配置属性，并使用 `@ConfigurationProperties` 注解。

```
@Configuration

@ConfigurationProperties(prefix = "spring.datasource")

public class DataSourceConfig {

    private String url;

    private String username;

    DataSourceProperties dataSourceProperties = new DataSourceProperties();
    dataSourceProperties.setUrl(attributes.get("url").toString());
    dataSourceProperties.setDataUsername(attributes.get("username").toString());
    dataSourceProperties.setDataPassword(attributes.get("password").toString());
    String driverClassName = attributes.get("driver-class-name") == null ?
    "com.zaxxer.hikari.HikariDataSource" : attributes.get("driver-class-name").toString();
    dataSourceProperties.setDriverClassName(driverClassName);
```

分库

```
public class DynamicDataSource extends AbstractRoutingDataSource {

    @Value("${mini-db-router.jdbc.datasource.default}")
    private String defaultDataSource;

    @Override
    protected Object determineCurrentLookupKey() {
    if(DBContextHolder.getDBKey()==null){
        return defaultDataSource;
    }else {
        return "db"+DBContextHolder.getDBKey();
    }
    }

}
```

`AbstractRoutingDataSource` 是 Spring 框架中提供的一个抽象类，用于实现基于运行时条件动态切换数据源的机制。它是实现多数据源路由的关键组件，允许应用程序根据特定的策略（如请求属性、用户角色、系统环境等）选择合适的数据库连接进行操作。

分表

```
@Intercepts({@Signature(type = Statement.class,method = "prepare",args =
{Connection.class,Integer.class})})
public class DynamicMybatisPlugin implements Interceptor {

    private Pattern pattern=Pattern.compile("(from|into|update)[\\s]{1,}(\\w{1,})");
    @Override
    public Object intercept(Invocation invocation) throws Throwable {
        // 获取StatementHandler
        StatementHandler statementHandler = (StatementHandler) invocation.getTarget();
        MetaObject metaObject = MetaObject.forObject(statementHandler,
SystemMetaObject.DEFAULT_OBJECT_FACTORY,
SystemMetaObject.DEFAULT_OBJECT_WRAPPER_FACTORY, new DefaultReflectorFactory());
        MappedStatement mappedStatement = (MappedStatement)
metaObject.getValue("delegate.mappedStatement");

        // 获取自定义注解判断是否进行分表操作
        String id = mappedStatement.getId();
        String className = id.substring(0, id.lastIndexOf("."));
        Class<?> clazz = Class.forName(className);
        DBRouterStrategy dbRouterStrategy = clazz.getAnnotation(DBRouterStrategy.class);
        if(null!=dbRouterStrategy||!dbRouterStrategy.splitTable()){
            return invocation.proceed();
        }

        // 获取SQL
        BoundSql boundSql = statementHandler.getBoundSql();
        String sql = boundSql.getSql();

        // 替换SQL表名 USER 为 USER_03
        Matcher matcher = pattern.matcher(sql);
        String tableName=null;
        if(matcher.find()){
            tableName=matcher.group().trim();
        }
        assert null!=tableName;
        String replaceSql = matcher.replaceAll(tableName + "_" + DBContextHolder.getTBKey());

        // 通过反射修改SQL语句
        Field field = boundSql.getClass().getDeclaredField("sql");
        field.setAccessible(true);
        field.set(boundSql,replaceSql);
        field.setAccessible(false);
        return invocation.proceed();
    }
}
```

StatementHandler 是 MyBatis 框架中的一个核心组件，属于其内部架构的一部分。它是 MyBatis 处理 SQL 语句的核心接口之一，负责 SQL 语句的预编译、参数设置、执行以及结果集的处理。**StatementHandler** 在 MyBatis 的执行过程中扮演着至关重要的角色，是执行 SQL、处理 JDBC **Statement**（包括 **PreparedStatement** 和 **CallableStatement**）的关键。

AOP

```
@Pointcut("@annotation(cn.bugstack.middleware.db.router.annotation.DBRouter)")
public void aopPoint() {
}
```

切点：注解

使用

引入依赖

```
<groupId>cn.bugstack.middleware</groupId>
<artifactId>db-router-spring-boot-starter</artifactId>
<version>1.0.2-SNAPSHOT</version>
```

yaml配置数据源信息

```
mini-db-router:
  jdbc:
    datasource:
      dbCount: 2
      tbCount: 4
      default: db00
      routerKey: userId
      list: db01,db02
      db00:
        driver-class-name: com.mysql.cj.jdbc.Driver
        url: jdbc:mysql://localhost:3306/big_market?
        useUnicode=true&characterEncoding=utf8&autoReconnect=true&zeroDateTimeBehavior=convertTo
        Null&serverTimezone=UTC&useSSL=true
        username: root
        password: 1
        type-class-name: com.zaxxer.hikari.HikariDataSource
      pool:
        pool-name: Retail_HikariCP
        minimum-idle: 15 #最小空闲连接数量
        idle-timeout: 180000 #空闲连接存活最大时间，默认600000（10分钟）
        maximum-pool-size: 25 #连接池最大连接数，默认是10
        auto-commit: true #此属性控制从池返回的连接的默认自动提交行为,默认值：true
        max-lifetime: 1800000 #此属性控制池中连接的最长生命周期，值0表示无限生命周期，默认1800000
        即30分钟
        connection-timeout: 30000 #数据库连接超时时间,默认30秒，即30000
        connection-test-query: SELECT 1
```


✓ sharding-jdbc

官方文档 <https://shardingsphere.apache.org/document/current/cn/quick-start/shardingsphere-jdbc-quick-start/>

数据分片

根据业务拆分的方式称为垂直分片，又称为纵向拆分，它的核心理念是专库专用

水平分片(根据某个字段分库分表)从理论上突破了单机数据量处理的瓶颈，并且扩展相对自由，是数据分片的标准解决方案

尽量使用本地事务，善于使用同库不同表可有效避免分布式事务带来的麻烦

分布式事务

ShardingSphere BASE 提供了分布式环境下，对数据最终一致性的保证

✓ Mycat

官网 <http://mycat.org.cn/>

mq

为了解耦流程, 项目中多次发送mq

生产者

发送mq消息

消费监听

收到mq消息,做处理

事务

项目中多次用了事务, 例如 聚合体相关就用事务

隔离类别

读未提交 问题例子: 转账

读已提交 两次读 中间有一个修改数据

可重复度 默认

串行化

事务传播机制

7种事务传播机制	
事务传播机制	描述
REQUIERD	默认, 如果没有当前事务, 就新建一个事务, 如果存在一个事务, 就加入到这个事务中
SUPPORTS	支持当前事务, 如果没有当前事务, 就以非事务方法执行
MANDATORY	使用当前事务, 如果没有当前事务, 就抛出异常
REQUIRED_NEW	新建事务, 如果存在当前事务, 就把当前事务挂起
NOT_SUPPORTED	以非事务方式执行操作, 如果当前事务存在则抛出异常
NESTED	如果当前存在事务, 则在事务内执行, 如果当前没有事务, 则执行与REQUIRED类似操作

编程式事务

本项目

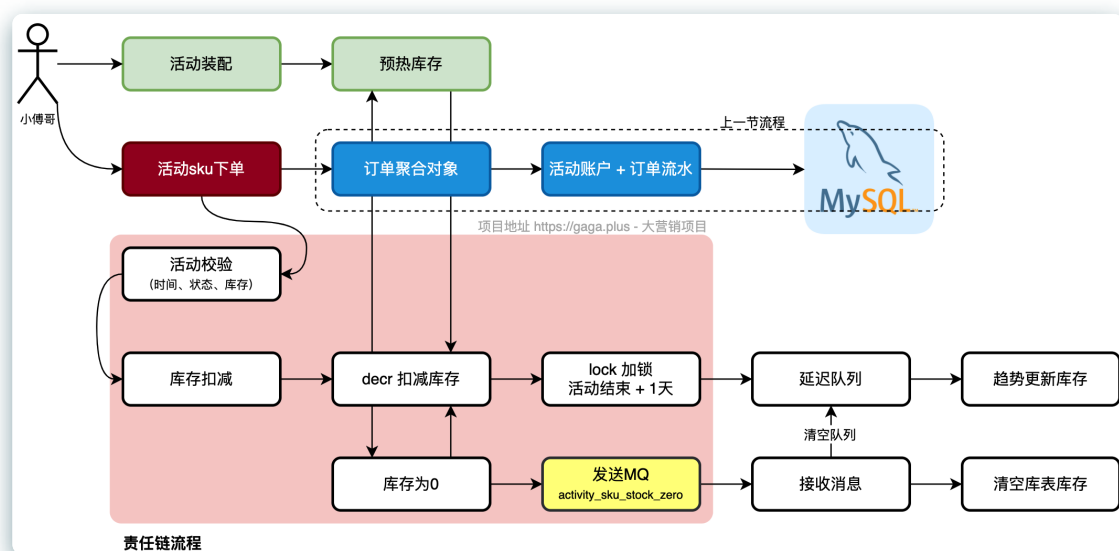
声明式事务

加注解 原理 aop 代理

失效情况

- 1.非public
- 2.事务隔离设置有误
- 3.事务传播设置有误
- 4.同一个类A调用B 最常见 因为 只有被另一个类调用, Spring才会代理

活动领域



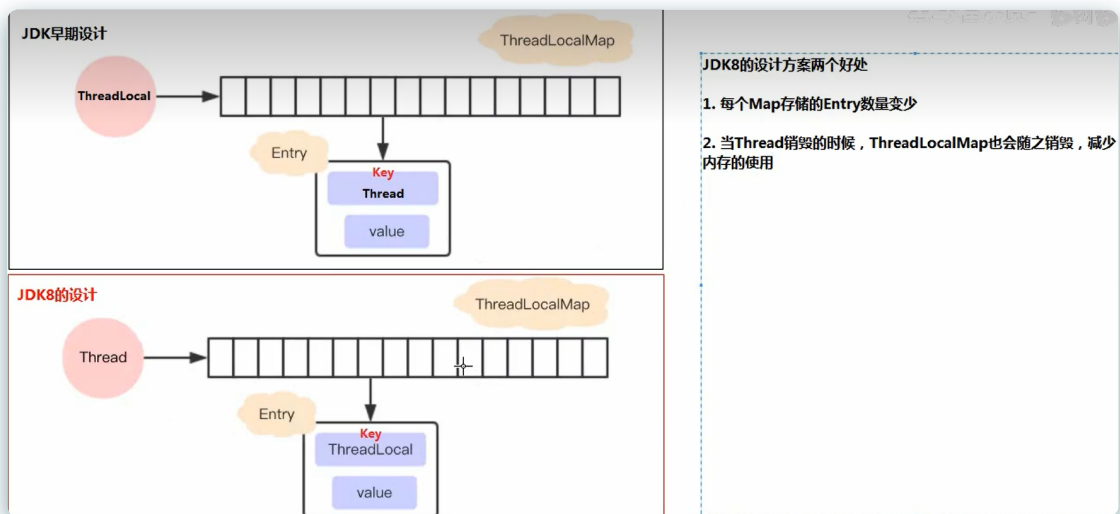
活动订单 下单一个sku

活动流水 用户参与活动流水

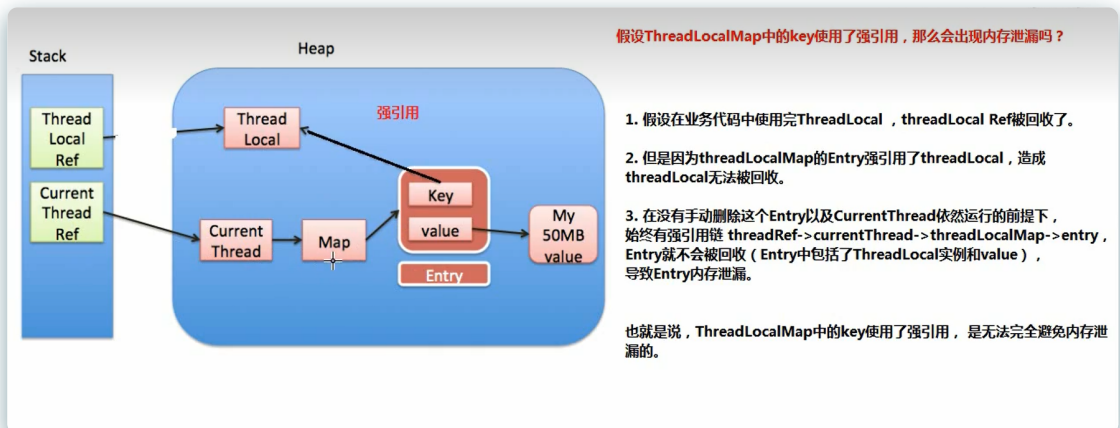
Redis客户端

ThreadLocal

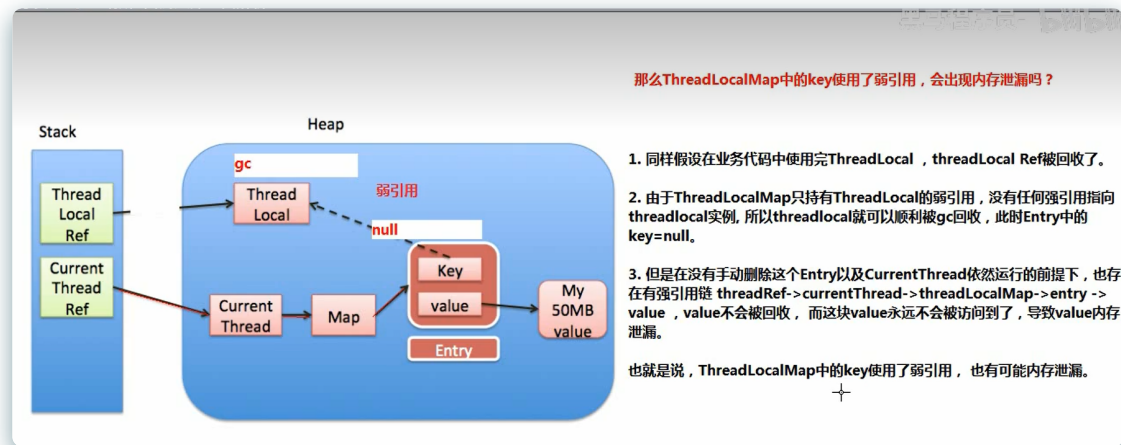
结构



强引用



弱引用



解决哈希冲突

id生成(lottery)

1、学习目标：

ID生成策略领域开发，包括雪花算法、日期拼接、随机数生成ID，

2、策略模式（行为模式的一种）

*使用策略模式对三种Id生成算法统一包装，由调用方决定使用哪种Id生成策略

-IdGenerator：定义获取Id的接口

*三种ID生成算法如下

-SnowFlake：雪花算法生成器，实现获取Id的接口，并将生成器交给Spring容器管理

-ShortCode：日期拼接Id生成器，获取当前的年、周、日、小时拼接

-RandomNumeric：随机Id生成器，用lang3中的工具实现

-IdContext：@Configuration将该类标记为配置类，@Bean标记idGenerator方法，该方法把三种Id生成策略实例化对象存入Map（这里有个疑问，是因为这三个算法策略都用@Component交给了Spring容器管理，所以，在IdContext中直接获取对象？）并将这个Map对象交给Spring IOC容器管理，调用方传入Id枚举值获取Id

3、雪花算法(重点,分布式id)

*首先如果使用MySQL数据库，则主键最好是有序的，不推荐使用UUID作为主键，因为MySQL在维护聚簇索引时是按照主键的顺序排序，即每个数据页中的数据一定按照主键从小到大排序，数据之间通过单项链表连接，数据页之间通过双向链表连接，如果主键是自增的，MySQL可以根据主键目录很快定位新增数据的插入位置，如果主键不自增，则每次都要从头开始寻找插入位置，效率不高

*引入雪花算法

雪花算法生成的ID是一个64位的整数，其结构可以分为以下几个部分：

1. **1位符号位**：始终为0，表示正数。
2. **41位时间戳**：表示从1970年1月1日以来的毫秒数，这个数值可以使用大约69年，直到2038年问题（对于32位系统）之前。
3. **10位机器标识码**：可以部署在1024个节点，通常前5位作为数据中心标识，后5位作为服务器标识。
4. **12位序列号**：每个节点每毫秒内可以生成4096个ID，用于解决同一毫秒内多个请求的竞争问题。

雪花算法的工作流程如下：

- 获取当前时间的时间戳，转换为毫秒级。
- 获取当前机器的标识码，通常是预先配置好的。
- 为每个请求生成一个序列号，如果在同一毫秒内有多次请求，则序列号递增。
- 将上述三部分按位拼接起来，形成最终的64位整数ID。

雪花算法的优势在于：

- **全局唯一**：时间戳、机器标识和序列号的组合确保了ID的全局唯一性。
- **趋势递增**：由于时间戳在前，生成的ID整体上是递增的，这有助于索引优化。
- **高性能**：大部分操作都是位操作，且在单个节点上生成ID，不需要进行分布式协调，因此性能非常高。
- **灵活的可扩展性**：通过调整机器标识位数，可以轻松支持更多的服务器或数据中心。

需要注意的是，在实际应用中，要根据具体业务需求和系统规模来调整雪花算法中的各部分位数分配，以达到最佳的适应性和性能。

缺点：依赖系统时间，如果系统时间被回调或改变，可能会出现id重复

状态模式(lottery)

1.AbstractState: 抽象类定义了7个活动状态声明, 包括活动编辑、提审、撤审、通过、拒绝、关闭、开启,

2.在event包中定义每个状态类, 继承AbstractState并实现7个活动判断, 如“提审”状态可以变为“撤销|通过|拒绝|关闭”, “提审”状态不可以转为“开启|提审|活动中”, 则针对这7种判断实现代码逻辑, 可转的状态则调用数据仓储服务修改数据库状态, 并返回成功提示, 不可转的状态, 则只给返回“不可操作”的提示。